

Coldex Instrumentation — Hardware Verification & Next Steps

Oct 7, 2026 · @Aiden

Summary

The app now recovers from missing or unplugged hardware by itself. The valve panel follows what the relay controller reports it is driving. New instruments are added as a driver file plus a YAML entry. Nothing needs to be edited in `run_app.py`. All of this is uncommitted on `claude_edits`. The 25 tests pass, and the GUI was smoke-tested headless in demo mode and in hardware mode with no Due attached. **Nothing has run on real hardware yet:** the tests below are what remains.

Area	What changed	Where
Relay firmware	Protocol v2: identity, status, release-all, error codes, strict parsing, pin readback, switch-panel hook, optional comms fail-safe	<code>firmware/relay_controls/</code>
Controller driver	Waits for the Due to boot and checks its identity; plain-English errors; polls status every 1 s; detects a dead link after 3 missed polls	<code>serial/pneumatic_communicator.py</code>
Connections	Failed or lost devices retry every 5 s instead of staying dead until restart; commands to an offline device fail at once	<code>hardware/device_manager.py</code>
Valve panel	A failed command restores the previous state. Status reports correct drift, for example after a Due reset, and log a warning.	<code>main_window.py</code> , <code>valve_panel_canvas.py</code>

Area	What changed	Where
Add-in structure	<code>driver:</code> / <code>demo_driver:</code> / <code>options:</code> in <code>devices.yaml</code> ; driver files auto-load; <code>LineSerialSensor</code> base class; MCP9601 thermocouple driver; template; step-by-step guide	<code>drivers/</code> , <code>docs/ADDING_A_DEVICE.md</code>
Launcher	<code>--demo</code> simulates every device and marks the window title; <code>--list-drivers</code>	<code>run_app.py</code>
Tools and tests	<code>tools/relay_bench.py</code> for manual relay tests; 23 new tests including a Python model of the firmware	<code>tools/</code> , <code>tests/</code>
Housekeeping	Simulated devices labelled in config; generated egg-info untracked; READMEs and changelog updated	<code>config/</code> , <code>*.md</code>

The panel still cannot show real valve position. "Open" means the controller is energizing that relay.

Before you flash: relay firmware protocol v2

Flash the new `firmware/relay_controls/relay_controls.ino` before running the app against real hardware. The app now asks the Due to identify itself on connect, and old firmware will fail that check. **The sketch has not been compiled:** this laptop has no Arduino SAM (Due) core and no C++ compiler. Expect to fix small compile errors in the Arduino IDE (Board: *Arduino Due (Programming Port)*).

Every message is framed as `<...>` and every reply is one line. Lines starting with `#` are notes the app ignores.

Send	Reply	Meaning
<code><n,s></code>	<code>1</code> or <code>0,<code></code>	Set relay <code>n</code> (0–23) to <code>s</code> (0 off, 1 on).

Send	Reply	Meaning
<?>	S,<states>,<modes>	24 characters each. States: 1 on, 0 off, E pin readback fault. Modes: A auto, M manual.
<I>	I,coldex_relay_controller,2,24	Identity, protocol version and channel count.
<X>	1	Release all relays now, ignoring the 100 ms lockout.

Error codes in 0,<code> : 1 malformed frame, 2 index out of range, 3 state not 0/1, 4 switched within 100 ms, 5 pin readback mismatch, 6 channel in MANUAL, 7 unknown command.

What the new checks can and cannot see:

- **Pin readback** reads the Due output pin after driving it. It catches a shorted or damaged pin, not a dead relay or a stuck valve.
- **Repeated commands** for the state a relay is already in now succeed without re-switching, so they are never hit by the lockout.
- **Status polling**: the app sends <?> every second. If the Due resets (brownout, USB replug), the panel updates to show every valve closed and logs a warning.
- **Stricter parsing**: values like 3x used to be accepted as 3. They are now rejected.

Hardware test checklist

Run tests 1–9 with the valve manifold's air supply off, or with the 24 V load side disconnected, so a mapping mistake cannot move anything. Use `python tools/relay_bench.py COM14` for manual commands, and close the dashboard first, since only one program can hold the COM port.

#	Test	How	Expected	Result
1	Compile and flash	Arduino IDE, Board: Arduino Due (Programming Port). Open <code>firmware/relay_controls/relay_controls.ino</code> and upload.	Compiles; fix any small errors I could not catch without a compiler.	Not run
2	Boot and identity	Serial Monitor at 9600. Press the Due reset button, then send <I> and <?>.	<pre># coldex_relay_controller booted... , then I,coldex_relay_controller,2,24 , then S, + 24 × 0 + , + 24 × A .</pre>	Not run

#	Test	How	Expected	Result
3	Relay map, all 24	Bench tool: <code>n,1</code> then <code>n,0</code> for $n = 1-24$.	Relay n on the expected module and channel clicks on, then off (see pin map in the firmware README).	Not run
4	Relays really release	Same as 3; watch each module's LED and listen for the release.	Every relay drops out on <code>n,0</code> . If one stays partly on, the 3.3 V HIGH level is not enough for that relay board.	Not run
5	Lockout	Serial Monitor: send <code><0,1><0,0></code> in one line.	<code>1</code> then <code>0,4</code> . Sending <code><0,0></code> again 1 s later gives <code>1</code> .	Not run
6	Readback with load	Bench tool: turn all 24 relays on, then <code>status</code> .	No <code>E</code> characters. An <code>E</code> means a pin cannot hold its level under the relay board's load.	Not run
7	App connects on start	<code>python run_app.py</code> with the Due plugged in.	Connectivity tile shows <i>Connected</i> within about 3 s. If it shows "No relay controller answered", raise <code>boot_wait_s</code> in <code>devices.yaml</code> .	Not run
8	Reset is detected	App running, valve 1 open. Press the Due reset button.	Within about 1 s the panel shows every valve closed and the log has a "Valve panel updated to match controller" warning.	Not run
9	Unplug and replug USB	App running. Unplug the Due, click a valve, then plug it back in.	Click is undone with a "did NOT change" status message; the tile shows the reason; it reconnects within about 5 s. Windows may assign a new COM number.	Not run
10	Valves match the panel (24 V and air on)	Click each valve in the panel once.	The physical valve that moves matches the label in <code>actuators.yaml</code> . <i>Close all</i> closes every unlocked valve.	Not run
11	Fail-safe, if you enable it	Set <code>COMMS_TIMEOUT_MS = 5000</code> and re-flash. Start the app, then end it in Task Manager.	All relays release about 5 s later.	Not run
12	Thermocouple, optional	Flash <code>thermocouple_reader</code> , enable <code>thermocouple</code> in <code>devices.yaml</code> , set its COM port. Unplug the probe.	Readings in °C appear; unplugging logs one "open circuit" warning.	Not run

If any test fails, note the reply text shown in the bench tool or log. That text names the error code.

Switch panel

The firmware is ready to block automated commands on any channel whose switch is out of AUTO. It stays switched off until you wire the sense inputs and set two constants. The app already reads the AUTO/MANUAL field from every status report and logs channels that are in MANUAL.

What the firmware assumes, which you can change:

- **One sense input per relay**, using the spare pole of each DPDT switch. The pin is set to `INPUT_PULLUP`.
- **AUTO = pole connects the pin to Due GND** (pin reads LOW). An open pin reads as MANUAL, so a broken sense wire fails safe: automation is blocked, not allowed.
- **3.3 V only**. A sense pin must never see 5 V or 24 V. Keep the sense pole electrically separate from the pole that switches the valve.

To turn it on, edit the top of `relay_controls.ino`:

1. Set `SWITCH_PANEL_INSTALLED = true`.
2. Fill `AUTO_SENSE_PINS[24]` with one Due pin per relay, in relay order, using `-1` for channels without a switch.
3. Re-flash. Then check that `<?>` shows `A` for switches in AUTO and `M` for the others, and that `<n,1>` on a MANUAL channel replies `0,6`.

Free pins that avoid the relay outputs (22–29, 32–39, 42–49), USB serial and I²C: 2–13, 30–31, 40–41, 50–53 and A0–A11. That is enough for 24 direct inputs. A board design that wants fewer wires to the Due could use an MCP23017 I²C expander instead; that would need a firmware change.

Open questions for the switch-panel design:

- ☐ Which switch position is AUTO, which is MANUAL ON, and which is OFF on the ON-OFF-ON toggles?
- ☐ Does OFF need to be sensed separately from MANUAL ON? The firmware currently only distinguishes AUTO from not-AUTO.
- ☐ Should a channel moved from MANUAL back to AUTO keep its relay state, or be released?

Decisions needed from you

The first decision is about safety: what "safe" means for each valve when the laptop or controller fails. The code currently assumes "all relays released". Whether that is safe depends on whether each valve is normally closed or normally open.

- ☐ **Is "all relays released" safe for every valve and crusher?** It happens at every Due power-up, at every reset, and every time the app opens the COM port. The crushers are configured with `default_active: true` (powered = raised), so a release lowers them. If any channel must hold its state through a reset, it needs hardware handling such as a latching relay or the switch panel.

- ☐ **Turn on the communication-loss fail-safe?** If the app crashes, the relays currently hold their last state indefinitely. Setting `COMMS_TIMEOUT_MS = 5000` releases them 5 s after the app goes quiet. Recommended once the answer above is yes.
- ☐ **Add a hardware watchdog?** The Due can reset itself if the firmware hangs, which releases every relay. I left this out until the fail-safe policy above is settled.
- ☐ **Keep the simulated `environment` device enabled?** It feeds fake ambient temperature and humidity into the dashboard and into recorded sessions. Set `enabled: false` in `config/devices.yaml` if recordings should contain only real data.
- ☐ **Confirm `COM14`** is the controller's port on the lab laptop. It changes between computers and sometimes after replugging.
- ☐ **Crusher tile state.** The valve panel now follows the controller's status reports, but the crusher tile does not yet. It only tracks its own command results. Worth doing once the crushers have relay numbers assigned.
- ☐ **Relay board input compatibility.** Tests 4 and 6 will show whether the 3.3 V Due outputs reliably drive and release the 5 V relay modules. If not, add a level shifter or transistor buffer per module.

Suggested next steps

The biggest wins are getting the vacuum gauges into the app and adding YAML-defined valve sequences. Both now fit the driver system added in this pass. Ordered by value to the lab:

#	Change	What it gives you	Effort
1	Promote the GRAPHIX and Adalogger code from <code>other/</code> into <code>drivers/</code>	GP475, PDR-D-1 and GRAPHIX pressures show up in the dashboard and session recordings. The reference code is already tested; it needs a thin driver wrapper and a <code>devices.yaml</code> block each.	Small

#	Change	What it gives you	Effort
2	YAML valve sequences ("recipes")	Non-programmers write steps like <code>open valve_03</code> , <code>wait 5 s</code> , <code>close valve_03</code> in a file and run them from a dashboard tile, with a stop button. This is the main path to automation without Python.	Medium
3	Communication-loss fail-safe in firmware	The Due puts relays in a defined safe state if the laptop stops talking (crash, cable pulled). Needs your safety decision first; see Decisions.	Small
4	Switch-panel status in the UI	Once the switch panel is wired, each valve shows AUTO or MANUAL and manual valves are greyed out. The firmware hook and protocol field already exist.	Small
5	Continuous integration	GitHub Actions runs <code>pytest</code> on every push, so a broken change is caught before it reaches the lab laptop.	Small
6	Repository tidy-up	Move the eight root <code>*_GUIDE.md</code> files into <code>docs/guides/</code> . Stop tracking Inventor <code>OldVersions/</code> backups, or move CAD to Git LFS. Delete the empty <code>doc/</code> and <code>services/</code> folders.	Small
7	LAUDA RP 290 E driver	Bath temperature and setpoint in the dashboard over Ethernet.	Medium
8	Double-click launcher	A <code>Start Coldex Dashboard.bat</code> on the lab laptop that activates the venv and runs the app. No terminal needed.	Small
9	Agilent 7890A APG Remote sync	GC start/ready signals through the Due. Blocked on the electrical interface design.	Large

#	Change	What it gives you	Effort